Lab Sheet (continued): Adding Search to the Students Table

Student details

Full name	Meliz Alyana Ingalla
Student number	2025-7876
Section	BSIS02
Date	November 3, 2026

Your unique values

Use **these exact values** wherever a task says *your first name* or *your last name*.

What	Your value	Example
Your full name	Meliz Alyana Ingalla	Juan Dela Cruz
Your first name	Meliz Alyana	Juan
Your student number	2025-7876	2026-0147
Your repository link (the one from the Form Validation lab)	https://github.com/yanaingalla-hash/school-app-ingalla	same as the last example

Before you start

This lab **continues the SchoolApp** you built in the Form Validation lab (Tasks 0 to 10). Use the **same repository and the same Codespace**:

1. Go to github.com > your repository > **Code** > **Codespaces** and open your Codespace.
2. In the Terminal, type `cd SchoolApp`.
3. Run the app with `dotnet run` and open **Students**. You should see your own row from the last lab.

Your Students table has no search yet. You will add it in two ways:

	Option A: Search with a button	Option B: Live search
How it works	You type, click **Search**, and the page reloads with the results	The table filters **while you type**, with no button and no reload
Who searches	The **server** asks the database	The **browser** hides rows that don't match
Files you change	Controller **and** view	View only

Do **Option A first**, then **Option B**, unless your teacher tells you to do only one.

Throughout: stop the app with **Ctrl+C** and run `dotnet run` again after every code change.

Task 11: Check your Students page and add test data

11.1 The controller

Open 📄 `Controllers/StudentsController.cs`. The `Index` method that the generator wrote looks like this:

```
// GET: Students
public async Task<IActionResult> Index()
{
    return View(await _context.Student.ToListAsync());
}
```

11.2 The view

Open 📄 `Views/Students/Index.cshtml`. Find the `<table class="table">`. Its first three columns are **Student No.**, **Full Name** and **Email Address**, and each row is made in a `@foreach (var item in Model)` loop.

11.3 Add 3 classmates

Run the app, open **Students > Create New**, and add these 3 students. Fill in every field your validation rules need (Confirm Email too).

Student No.	Full Name	Email Address	Age
2026-0201	Maria Santos	maria.santos@school.edu	19
2026-0202	Pedro Reyes	pedro.reyes@school.edu	20
2026-0203	Carlo Mendoza	carlo.m@school.edu	18

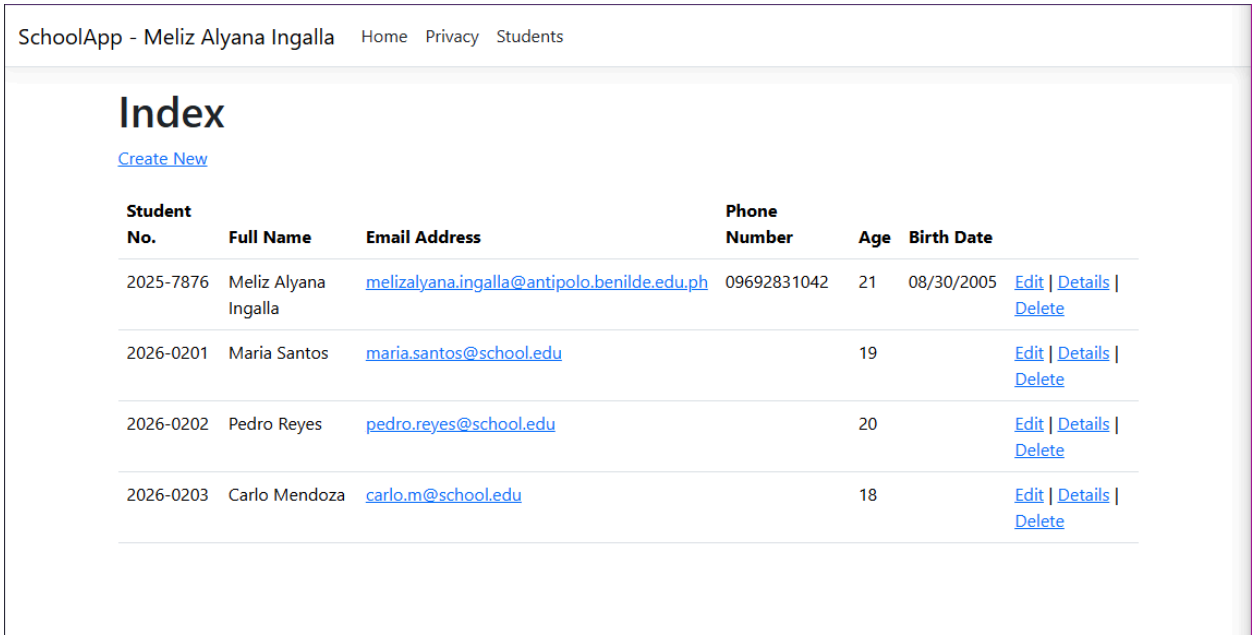
Your own row from the last lab must still be in the table.

Answer

Question	Your answer
How many students are in your table now?	4

Question	Your answer
Which validation message did you see, if any, while adding them?	None

Screenshot 11: the Students table with your own row and the 3 classmates, with your name in the menu bar.



Task 12 (Option A): Search with a button

12.1 Change the controller

📄 `Controllers/StudentsController.cs`: replace the whole `Index` method with this:

```
// GET: Students?searchString=maria
// searchString = what the user typed in the search box
public async Task<IActionResult> Index(string? searchString)
{
    // Start with all students, but don't load them yet
    var students = from s in _context.Student select s;

    // If the user typed something, keep only the matching students
    if (!string.IsNullOrEmpty(searchString))
    {
        // Lowercase, so "MARIA" also finds "Maria Santos"
        var text = searchString.ToLower();

        // Match the name, the student number OR the email
        students = students.Where(s => s.Name!.ToLower().Contains(text)
                                    || s.StudentNo!.ToLower().Contains(text)
                                    || s.Email!.ToLower().Contains(text));
    }

    // Send the text back, so it stays in the search box
```

```
    ViewData["searchString"] = searchString;

    // Now run the search and show the results
    return View(await students.ToListAsync());
}
```

12.2 Add the search form to the view

📄 `Views/Students/Index.cshtml`: add this right **above** `<table class="table">`. In the placeholder, replace `Juan` with **your first name**.

```
@* Option A: search with a button *@
@* method="get" puts the search text in the address bar *@
<form asp-action="Index" method="get" class="mb-3">
    @* name="searchString" must match the controller's parameter name *@
    <input type="text" name="searchString" value="@ViewData["searchString"]"
           placeholder="Name, student no. or email - Juan" />
    <button type="submit" class="btn btn-primary">Search</button>
    @* Clear goes back to /Students with no search, so all rows show *@
    <a asp-action="Index" class="btn btn-secondary">Clear</a>
</form>
```

12.3 Test it

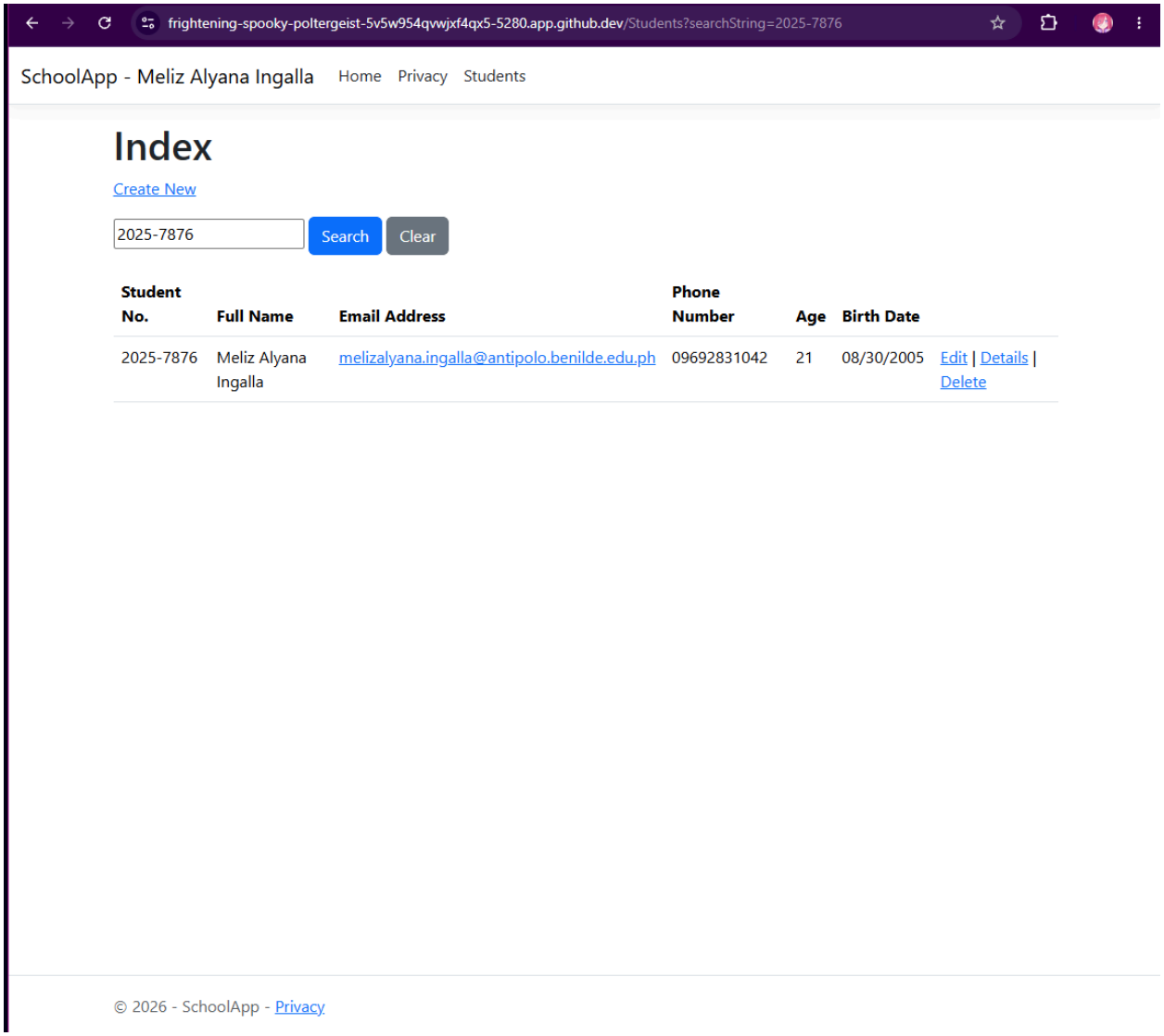
Save, stop the app and run it again. Open **Students**, type each value below, and click **Search**.

Answer

Type this	Students shown (write the names)
your first name	Meliz Alyana Ingalla
your student number	Meliz Alyana Ingalla
`MARIA`	Maria Santos
`2026-020`	Maria Santos Pedro Reyes Carlo Mendoza
`school.edu`	Maria Santos Pedro Reyes Carlo Mendoza
`xyz`	
(click **Clear**)	Meliz Alyana Ingalla Maria Santos Pedro Reyes Carlo Mendoza

Question	Your answer
After searching your first name, what is the full address in the address bar?	https://frightening-spooky-poltergeist-5v5w954qvwjxf4qx5-5280.app.github.dev/Students?searchString=meliz+alyana
Did the page reload when you clicked Search? (Yes / No)	Yes
Which line in the controller makes the search match the email too? (copy it)	|| s.Email!.ToLower().Contains(text));

Screenshot 12: the Students page after searching **your student number**, showing the address bar and only your row.



Task 13 (Option B): Live search (no button)

The controller stays as it is after Task 12. Everything happens in 📄 `Views/Students/Index.cshtml`.

13.1 Replace the search form with one box

Delete the whole `<form>...</form>` from Task 12 (skip this if you didn't do Option A) and put this in its place. Replace `Juan` with **your first name**:

```
@* Option B: live search. No form and no button: it filters while you type *@
<input type="text" id="searchBox" class="form-control mb-3"
       placeholder="Live search - Juan" />
```

13.2 Give the table body an id

In the same file, change `<tbody>` to:

```
<tbody id="studentRows">
```

13.3 Add the script

Add this at the very **bottom** of the file:

```
@section Scripts {
<script>
    // Runs every time a key is typed in the search box
    document.getElementById("searchBox").addEventListener("input", function () {
        // What the user typed, in lowercase so "MARIA" matches "Maria"
        const text = this.value.toLowerCase();

        // Look at every row in the table
        document.querySelectorAll("#studentRows tr").forEach(function (row) {
            // Student No. (cell 0), Full Name (cell 1) and Email (cell 2) together
            const rowText = (row.cells[0].textContent + " " +
                             row.cells[1].textContent + " " +
                             row.cells[2].textContent).toLowerCase();

            // Show the row if it contains the text, hide it if not
            row.style.display = rowText.includes(text) ? "" : "none";
        });
    });
</script>
}
```

If you see the error "Section 'Scripts' is already defined": your file already has a `@section Scripts { ... }`. Put only the `<script>...</script>` part inside the one you have.

13.4 Test it

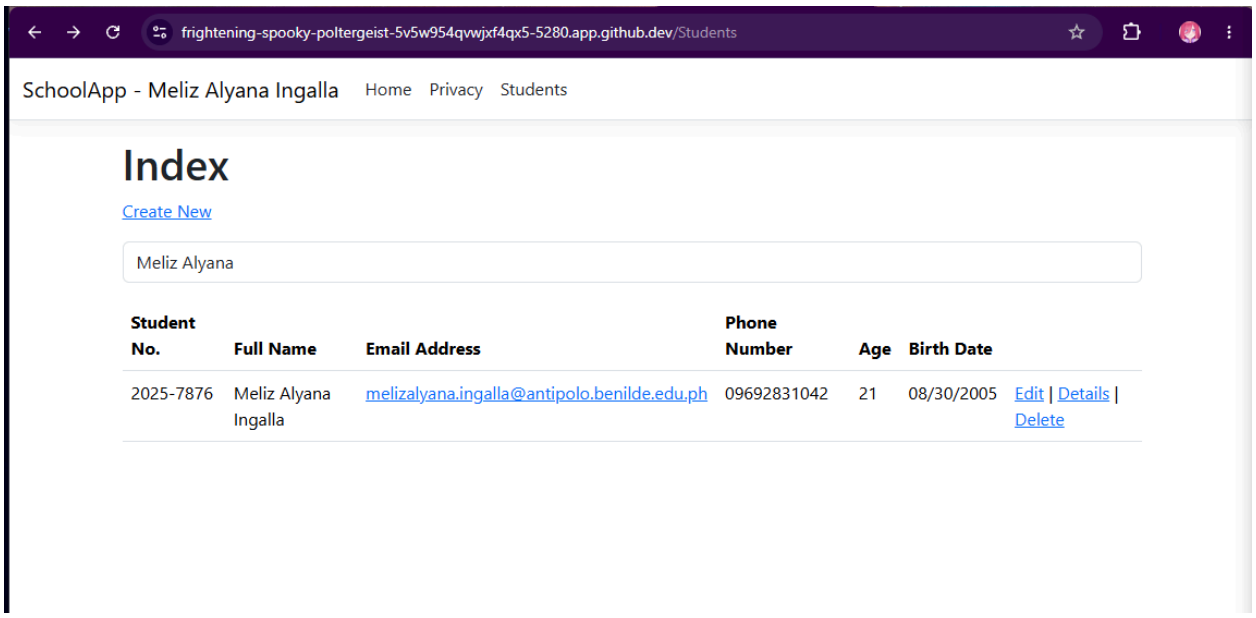
Save, stop the app and run it again. Open **Students** and type **one letter at a time**. Don't press Enter.

Answer

Type this	Students shown (write the names)
`m`	Meliz Alyana Ingalla Maria Santos Carlo Mendoza
`ma`	Maria Santos
`mar`	Maria Santos
your first name	Meliz Alyana Ingalla
your student number	Meliz Alyana Ingalla
(delete everything in the box)	Meliz Alyana Ingalla Maria Santos Pedro Reyes Carlo Mendoza

Question	Your answer
Did the address bar change while you typed? (Yes / No)	No
Did the page reload while you typed? (Yes / No)	No
Why do we search `cells[0]`, `cells[1]` and `cells[2]` and not the whole row? (Hint: look at the Edit, Details and Delete links)	Because Edit, Details, and Delete are part of the row but not considered student information.

Screenshot 13: the Students page while **your first name** is typed in the live search box, showing only your row.



Task 14: Compare and save your work

14.1 Compare the two options

Question	Your answer
Which option asks the database every time you search?	Option A
Which option works without reloading the page?	Option B
The school has 10,000 students. Which option is better, and why?	Option A, because it doesn't show all of the results which is better for a database with 10,000 students

Question	Your answer
	so that it wouldn't overload the page.
You want to send a classmate a link to the search results. Which option lets you do that, and why?	Option A, because the search string result is included in the address link.

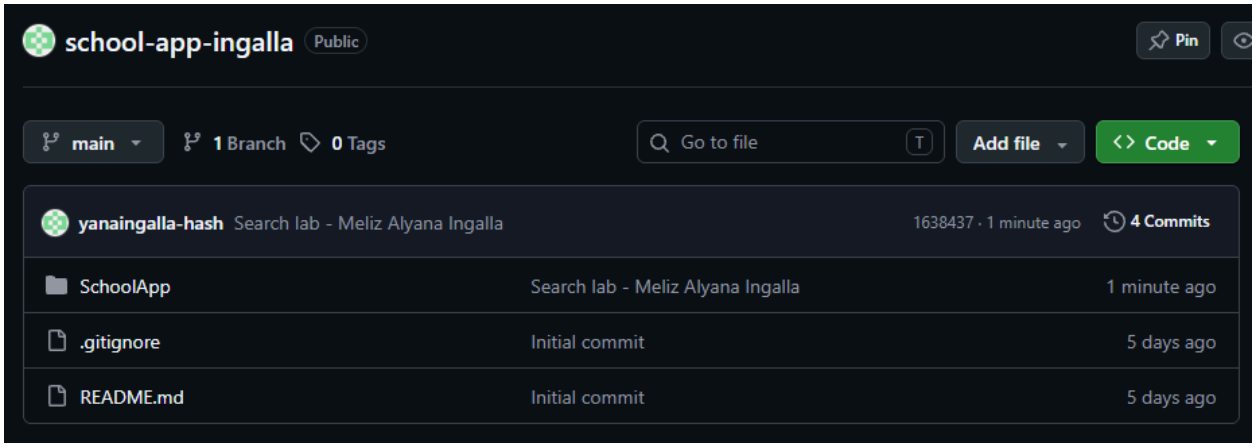
14.2 Push to GitHub

Stop the app (**Ctrl+C**), then run these lines one at a time. Replace `Juan Dela Cruz` with **your full name**:

```
git add Controllers/StudentsController.cs Views/Students/Index.cshtml
git commit -m "Search lab - Juan Dela Cruz"
git push
```

Question	Your answer
The commit message shown on GitHub	Search lab - Meliz Alyana Ingalla

Screenshot 14: your repository's commit history on GitHub, showing **both** your Form Validation commit and this Search commit.



Submission checklist

- ☐ Student details and **Your unique values** are filled in
- ☐ Every **Answer** box in Tasks 11 to 14 is filled in
- ☐ Screenshots 11, 12, 13 and 14 are attached, and your name is visible in the menu bar
- ☐ The search box placeholder has your first name
- ☐ Your code is pushed to the **same** repository as the Form Validation lab

What your teacher will check

Check	Where
`Index(string? searchString)` searching Name, Student No. and Email	`Controllers/StudentsController.cs`
The live search script and `id="studentRows"`	`Views/Students/Index.cshtml`
Your first name in the placeholder	`Views/Students/Index.cshtml` and Screenshot 13
Your own row found by your student number	Screenshot 12
Two commits with your full name, in the same repository	Your repository's commit history